

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

AIM

To evaluate the scalability of Merge Sort in a simulated parallel (multi-threaded) environment by measuring execution time for different thread counts, calculating speedup, and interpreting whether parallelism improves performance.

PROCEDURE

1. Generate a random dataset of size 100000.
2. Implement Sequential Merge Sort using the standard merge-sort recursion and merge function.
3. Implement Parallel Merge Sort using ThreadPoolExecutor to process left and right halves concurrently up to a specified recursion depth (max_depth).
4. Measure sequential execution time (seq_time).
5. For thread levels [1, 2, 4], run the parallel merge sort, record execution time (par_time), and compute speedup:
 $\text{Speedup} = \text{seq_time} / \text{par_time}.$

6. Compare execution times and speedup values to analyze scalability and suitability of Merge Sort for parallel processing in this environment.

CODE

```
import random
import time
from concurrent.futures import ThreadPoolExecutor

# Merge Function
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Sequential Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

# Parallel Merge Sort
def parallel_merge_sort(arr, depth=0, max_depth=2):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    if depth >= max_depth:
        return merge(
            parallel_merge_sort(arr[:mid], depth+1, max_depth),
            parallel_merge_sort(arr[mid:], depth+1, max_depth)
        )
```

```

with ThreadPoolExecutor(max_workers=2) as executor:
    left = executor.submit(parallel_merge_sort, arr[:mid], depth+1, max_depth)
    right = executor.submit(parallel_merge_sort, arr[mid:], depth+1, max_depth)
    return merge(left.result(), right.result())

```

Generate Dataset

```
data = [random.randint(1, 1000000) for _ in range(100000)]
```

Sequential Time

```

start = time.time()
merge_sort(data.copy())
seq_time = time.time() - start
print("Sequential Time:", round(seq_time, 4), "seconds")

```

Simulated Thread Levels

```

for threads in [1, 2, 4]:
    start = time.time()
    parallel_merge_sort(data.copy(), max_depth=threads)
    par_time = time.time() - start

```

```

speedup = seq_time / par_time
print("-----")
print("Threads :", threads)
print("Execution Time :", round(par_time, 4), "seconds")
print("Speedup :", round(speedup, 2))

```

OUTPUT

The screenshot shows an online assessment interface titled "ASSESSMENT-CO3/AT1-EXPERIMENTS". The language is set to Python. The question is "Q1 Merge Sort" (10 marks). The question text asks to evaluate the scalability of Merge Sort in parallel computing environments by simulating multi-threaded execution. The "Your Code" section contains the following Python code:

```

1 import random
2 import time
3 from concurrent.futures import ThreadPoolExecutor
4
5 # Merge Function
6
7 def merge(left, right):
8     result = []
9     i = j = 0
10
11     while i < len(left) and j < len(right):
12         if left[i] < right[j]:
13             result.append(left[i])
14             i += 1
15         else:
16             result.append(right[j])
17             j += 1
18
19     result.extend(left[i:])
20     result.extend(right[j:])
21     return result

```

The "Input" section contains the text "stdin input". The "Output" section displays the following results:

```

Sequential Time: 0.3437 seconds
Threads : 1
Execution Time : 0.433 seconds
Speedup : 0.74

```

RESULT

Sequential Time: 0.3437 seconds

Parallel results:

- Threads = 1 → Execution Time = **0.4530 s**, Speedup = **0.76**
- Threads = 2 → Execution Time = **0.4061 s**, Speedup = **0.85**
- Threads = 4 → Execution Time = **0.3905 s**, Speedup = **0.88**

Interpretation (brief):

As thread count increases (1 → 4), execution time slightly decreases, but speedup remains < 1 , meaning the parallel version is still slower than sequential in this Python thread-based simulation due to overhead and limited effective parallelism.

RUBRICS FOR CALCULATION

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____